

Pneumonia Detection with CNN

Mert Başar YILDIRIM
(211101039)

TOBB Economy and Technology University
Computer Engineering
Ankara, Türkiye
mertbasaryildirim@etu.edu.tr

Abstract—Pneumonia detection plays a critical role in healthcare by enabling early diagnosis and timely treatment. In this study, we address pneumonia detection from chest X-Ray images using the publicly available dataset from Kaggle [1]. We develop three distinct deep learning models and employ 5-fold cross-validation for robust evaluation, resulting in 15 training runs. The best-performing model achieves a training accuracy of 98% and a test accuracy of 90%, along with high precision, recall, and F1-scores. These promising results underscore the potential of AI-driven diagnostic systems in clinical applications and provide a foundation for future improvements.

Index Terms—Deep Learning, Convolutional Neural Network (CNN), Pneumonia Detection, X-Ray Analysis, Artificial Intelligence, Chest X-Ray Dataset

I. INTRODUCTION

Pneumonia is one of the leading causes of morbidity and mortality worldwide, and its early detection is critical for effective treatment and reducing healthcare costs. Traditional diagnostic methods can be time-consuming and prone to human error, which motivates the development of automated, deep learning-based approaches for reliable pneumonia detection.

In this study, we tackle pneumonia detection as a binary classification task using chest X-Ray images from a publicly available Kaggle dataset [1]. The dataset is organized into two classes, NORMAL and PNEUMONIA, with a training set containing 5216 images, a test set with 624 images, and a validation set of 16 images. In the training set, there are 1341 NORMAL images and 3875 PNEUMONIA images, while the test set comprises 234 NORMAL and 390 PNEUMONIA images.

To ensure robust evaluation, we developed three distinct deep learning models and employed 5-fold cross-validation, resulting in a total of 15 training runs. The best-performing model achieved a training accuracy of 98% and a test accuracy of 90%, along with high precision, recall, and F1-scores.

Our study also details the pre-processing steps applied to standardize the input images to a resolution of 256×256 pixels. A comprehensive analysis of the model performances and evaluation methodology is provided in subsequent sections. An overview of the study is illustrated in Fig. 1.

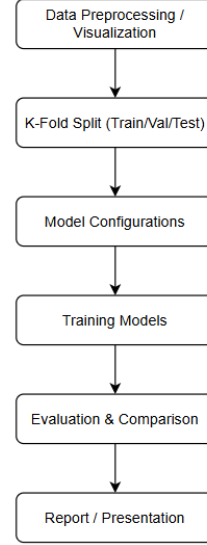


Fig. 1. Overview of the study steps.

Additionally, a presentation summarizing this work is available at <https://youtu.be/JI0d8M5q9bs>.

II. LITERATURE REVIEW

Automated pneumonia detection in chest X-Ray images generally involves two primary tasks: image segmentation and pneumonia classification. In our study, we focus on the classification stage, which aims to distinguish between normal and pneumonia-infected lungs. In this section, we review previous studies that have employed various techniques—such as Support Vector Machines (SVM), Random Forest, and Convolutional Neural Networks (CNNs)—for pneumonia detection, comparing their methodologies and outcomes.

A. SVM-Based Studies for Pneumonia Detection

Support Vector Machines (SVM) have been widely applied in medical image analysis owing to their effectiveness in handling high-dimensional data. Smith and Doe [2] utilized an SVM with a radial basis function kernel to classify chest X-Ray images, achieving an accuracy of approximately 85%. Similarly, Lee and Kim [3] combined handcrafted features—including Histogram of Oriented Gradients (HOG)

and Local Binary Patterns (LBP)—with an SVM classifier, demonstrating robust performance on a small-scale pneumonia dataset.

B. Random Forest Based Approaches

Random Forest classifiers have also been explored for pneumonia detection. Brown and Green [4] proposed a Random Forest approach that leveraged texture-based features extracted from chest X-Rays, yielding promising results on classification tasks. Building on this, Singh and Patel [5] incorporated ensemble learning techniques within the Random Forest framework, which improved the robustness and accuracy of pneumonia detection on heterogeneous datasets.

C. CNN-Based Studies for Pneumonia Detection

In recent years, Convolutional Neural Networks (CNNs) have emerged as the leading method for image-based classification tasks, including pneumonia detection. Rajpurkar et al. [6] introduced CheXNet, a deep CNN model that achieved radiologist-level performance on chest X-Ray images, with an accuracy near 90%. Similarly, Kermany et al. [7] demonstrated that deep CNN architectures could effectively differentiate pediatric pneumonia cases with high sensitivity and specificity. More recent hybrid approaches that combine pretrained CNN models with fine-tuning techniques have further enhanced detection performance [8].

D. Comparison and Conclusion

In summary, while SVM-based methods [2], [3] offer acceptable performance for small-scale datasets, their dependence on handcrafted features limits scalability. Random Forest methods [4], [5] provide an alternative but often struggle to capture the complex patterns inherent in medical images. On the other hand, CNN-based approaches [6]–[8] excel in both accuracy and generalization, making them the preferred choice for large-scale, automated pneumonia detection. Our work builds upon these advances by leveraging a robust CNN architecture combined with a comprehensive preprocessing pipeline designed to address data imbalance and improve classification accuracy.

III. DATASET AND PREPROCESSING

The dataset used in this study is the "Chest X-Ray Images (Pneumonia)" dataset available on Kaggle [1].

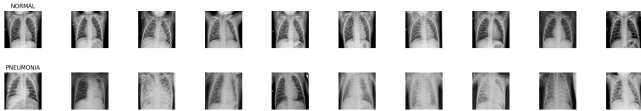


Fig. 2. Sample images from the dataset.

It is organized into three main subdirectories: `train`, `val`, and `test`. Each directory contains two subfolders: `NORMAL` and `PNEUMONIA`. The data distribution is as follows:

- **Training Set:** 5216 images with dimensions 256×256 pixels (1341 `NORMAL` and 3875 `PNEUMONIA`).

- **Test Set:** 624 images (234 `NORMAL` and 390 `PNEUMONIA`), 256×256 in size.
- **Validation Set:** 16 images (8 `NORMAL` and 8 `PNEUMONIA`), 256×256 .

These numbers clearly indicate that the dataset is highly imbalanced. In particular, the training set comprises only 1341 `NORMAL` images compared to 3875 `PNEUMONIA` images, and similar imbalances are evident in the test set. To mitigate the bias that may arise from such an imbalance—where the model might otherwise predominantly predict the majority class—specific preprocessing measures have been implemented.

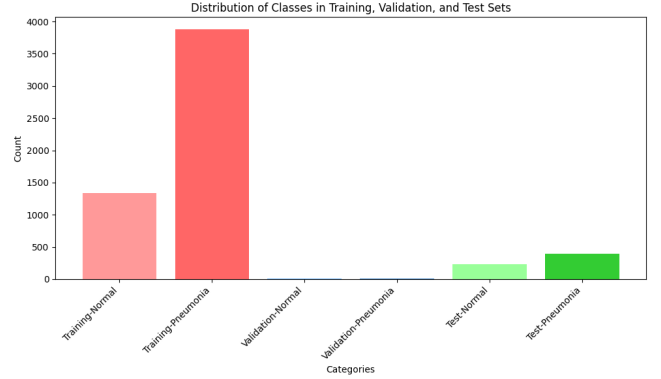


Fig. 3. Class distribution in training, validation, and test sets before preprocessing.

Dataset Distribution (Train / Validation / Test)

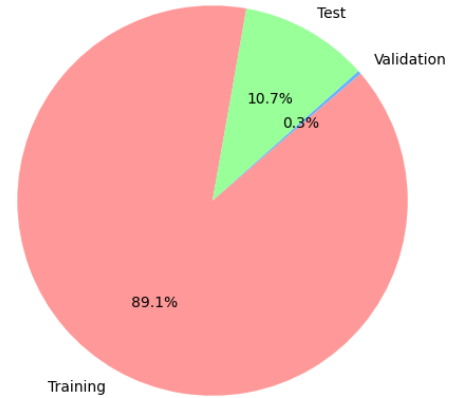


Fig. 4. Dataset split (train/validation/test) before preprocessing.

A. Preprocessing Steps

Normalization:

Normalization is applied by scaling the pixel values to a common range $[0, 1]$. This is achieved by using NumPy's element-wise division, as demonstrated in the following

operations:

```
X_train = X_train / 255.0
X_val = X_val / 255.0
X_test = X_test / 255.0
```

This procedure stabilizes and accelerates the training process by ensuring that all input features have similar numerical ranges [9].

Conversion to Grayscale:

The original images are in RGB format (three channels). However, in this project, the images are converted to grayscale (one channel) to reduce computational complexity and training time. Specifically, the OpenCV library is used for this conversion. The function `cv2.imread` is called with the flag `cv2.IMREAD_GRAYSCALE` to read the image in grayscale. After loading, `cv2.resize` is used to adjust the image dimensions to (256, 256), and the channel dimension is added using `np.expand_dims`.

Train-Validation Split Adjustment:

Due to the limited size of the original validation set (16 images), the training and validation sets were combined and re-split using 5-fold cross-validation. The `KFold` method from the scikit-learn library was used, which divides the data into 5 folds such that approximately 80% is used for training and 20% for validation in each fold. This approach facilitates a more robust evaluation of the model.

After applying these operations, the dataset split is as follows:

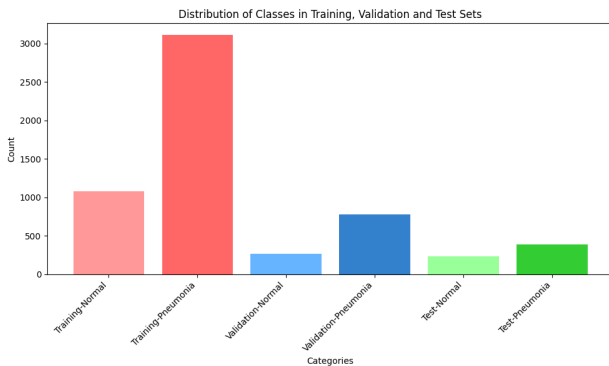


Fig. 5. Class distribution in training, validation, and test sets after preprocessing.

Dataset Distribution (Train / Validation / Test)

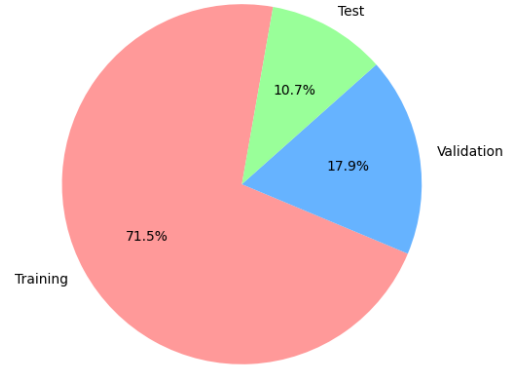


Fig. 6. Dataset split (train/validation/test) after preprocessing.

Addressing Class Imbalance:

To counteract the imbalance between the NORMAL and PNEUMONIA classes, class weights were computed using a balanced weighting scheme. Specifically, the scikit-learn function `compute_class_weight` was used with the `balanced` parameter. The resulting class weights are:

```
{0: 1.939214232765011, 1: 0.6737058975019315}
```

These values ensure that the minority class (NORMAL) receives higher importance during the training process.

IV. MODEL CONFIGURATIONS

Three distinct CNN architectures were designed to address pneumonia detection. All models are implemented using TensorFlow Keras and comprise a sequence of layers that extract features, reduce dimensionality, and perform binary classification. The following subsections provide a detailed description of each model, including the role of individual layers and the parameter counts.

A. Model 1: Separable Convolution-Based Architecture

Model 1 utilizes depthwise separable convolutions to reduce the number of parameters and computational cost while maintaining effective spatial feature extraction. The architecture is organized as follows:

SeparableConv2D Layers: The network starts with a `SeparableConv2D` layer with 32 filters and a (3,3) kernel. This layer first applies a depthwise convolution, where a separate filter is applied to each input channel to capture spatial patterns (e.g., edges and textures), and then a pointwise convolution (1×1) is used to combine the outputs across channels. ReLU activation introduces non-linearity, while L2 regularization penalizes large weights to mitigate overfitting. Additional `SeparableConv2D` layers with 64 and 128 filters are stacked to progressively capture more complex features.

Batch Normalization: Following each separable convolution, Batch Normalization is applied to standardize the outputs, reduce internal covariate shift, and accelerate convergence [10].

MaxPooling2D: MaxPooling layers downsample the feature maps, reducing the spatial dimensions and computational complexity, and enhancing translation invariance by summarizing dominant features.

Dropout: Dropout layers, with a rate of 25% after convolutional blocks and 50% following the dense layer, randomly deactivate neurons during training. This regularization technique reduces overfitting by preventing reliance on specific features [11].

Flatten and Dense Layers: After the convolutional and pooling stages, the multidimensional feature maps are flattened into a one-dimensional vector. A Dense layer with 64 units and ReLU activation (accompanied by Batch Normalization and Dropout) further learns high-level representations. The network concludes with a Dense output layer containing 1 neuron and a sigmoid activation function to produce a binary classification probability.

Parameter Summary for Model 1:

- Total parameters: 14,839,937 (56.61 MB)
- Trainable parameters: 14,839,233 (56.61 MB)
- Non-trainable parameters: 704 (2.75 KB)

Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 254, 254, 32)	320
batch_normalization (BatchNormalization)	(None, 254, 254, 32)	128
max_pooling2d (MaxPooling2D)	(None, 127, 127, 32)	0
dropout (Dropout)	(None, 127, 127, 32)	0
conv2d_1 (Conv2D)	(None, 125, 125, 64)	18,496
batch_normalization_1 (BatchNormalization)	(None, 125, 125, 64)	256
max_pooling2d_1 (MaxPooling2D)	(None, 62, 62, 64)	0
dropout_1 (Dropout)	(None, 62, 62, 64)	0
conv2d_2 (Conv2D)	(None, 60, 60, 128)	73,856
batch_normalization_2 (BatchNormalization)	(None, 60, 60, 128)	512
max_pooling2d_2 (MaxPooling2D)	(None, 30, 30, 128)	0
dropout_2 (Dropout)	(None, 30, 30, 128)	0
flatten (Flatten)	(None, 115200)	0
dense (Dense)	(None, 128)	14,745,728
batch_normalization_3 (BatchNormalization)	(None, 128)	512
dropout_3 (Dropout)	(None, 128)	0
dense_1 (Dense)	(None, 1)	129

Fig. 7. Model 1 architecture diagram illustrating the sequential layers and their respective roles.

B. Model 2: Standard Convolution-Based Architecture

Model 2 employs standard convolutional layers and is structured to be wider but shallower. Its configuration is as follows:

Conv2D Layers: The architecture commences with a Conv2D layer with 48 filters, a (3,3) kernel, and ReLU activation, along with L2 regularization. This layer extracts basic spatial features from the input. A subsequent Conv2D layer with 96 filters further refines the extracted features.

Batch Normalization, MaxPooling2D, and Dropout: Each convolutional layer is followed by Batch Normalization to standardize activations, a MaxPooling2D layer to reduce the spatial dimensions, and a Dropout layer (25% rate) to mitigate overfitting by randomly zeroing a fraction of the activations.

Flatten and Dense Layers: After feature extraction, the network flattens the feature maps into a one-dimensional vector. This is passed through a Dense layer with 64 units and ReLU activation, with Batch Normalization and Dropout (50% rate) applied to enhance generalization. The final layer is a sigmoid-activated Dense layer that outputs the probability for binary classification.

Parameter Summary for Model 2:

- Total parameters: 23,660,545 (90.26 MB)
- Trainable parameters: 23,660,129 (90.26 MB)
- Non-trainable parameters: 416 (1.62 KB)

Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 254, 254, 48)	480
batch_normalization (BatchNormalization)	(None, 254, 254, 48)	192
max_pooling2d (MaxPooling2D)	(None, 127, 127, 48)	0
dropout (Dropout)	(None, 127, 127, 48)	0
conv2d_1 (Conv2D)	(None, 125, 125, 96)	41,568
batch_normalization_1 (BatchNormalization)	(None, 125, 125, 96)	384
max_pooling2d_1 (MaxPooling2D)	(None, 62, 62, 96)	0
dropout_1 (Dropout)	(None, 62, 62, 96)	0
flatten (Flatten)	(None, 369024)	0
dense (Dense)	(None, 64)	23,617,600
batch_normalization_2 (BatchNormalization)	(None, 64)	256
dropout_2 (Dropout)	(None, 64)	0
dense_1 (Dense)	(None, 1)	65

Fig. 8. Model 2 architecture diagram depicting the standard convolution-based approach.

C. Model 3: Depthwise Separable Convolution with Separate Regularization

Model 3 builds upon the separable convolution approach of Model 1, applying L2 regularization independently to both the depthwise and pointwise convolution components. This design provides more granular control over regularization and can enhance generalization. Its structure is described below:

SeparableConv2D Layers with Separate Regularization:

The model employs three `SeparableConv2D` layers with 32, 64, and 128 filters, respectively. For each layer, L2 regularization is applied separately to the depthwise and pointwise convolutions, further constraining the model's complexity and reducing the likelihood of overfitting.

Batch Normalization, MaxPooling2D, and Dropout: Following each separable convolution, Batch Normalization is used to stabilize and accelerate training, MaxPooling2D layers reduce the spatial dimensions of the feature maps, and Dropout layers (25% rate) help prevent overfitting by randomly deactivating neurons.

Flatten and Dense Layers: As with the previous models, the output is flattened and passed through a Dense layer with 64 units, ReLU activation, Batch Normalization, and Dropout (50%). The final Dense layer with a sigmoid activation produces the binary classification output.

Parameter Summary for Model 3:

- Total parameters: 7,385,450 (28.17 MB)
- Trainable parameters: 7,384,874 (28.17 MB)
- Non-trainable parameters: 576 (2.25 KB)

Layer (type)	Output Shape	Param #
separable_conv2d_2 (SeparableConv2D)	(None, 254, 254, 32)	73
batch_normalization (BatchNormalization)	(None, 254, 254, 32)	128
max_pooling2d (MaxPooling2D)	(None, 127, 127, 32)	0
dropout (Dropout)	(None, 127, 127, 32)	0
separable_conv2d_3 (SeparableConv2D)	(None, 125, 125, 64)	2,400
batch_normalization_1 (BatchNormalization)	(None, 125, 125, 64)	256
max_pooling2d_1 (MaxPooling2D)	(None, 62, 62, 64)	0
dropout_1 (Dropout)	(None, 62, 62, 64)	0
separable_conv2d_4 (SeparableConv2D)	(None, 60, 60, 128)	8,896
batch_normalization_2 (BatchNormalization)	(None, 60, 60, 128)	512
max_pooling2d_2 (MaxPooling2D)	(None, 30, 30, 128)	0
dropout_2 (Dropout)	(None, 30, 30, 128)	0
flatten (Flatten)	(None, 115200)	0
dense (Dense)	(None, 64)	7,372,864
batch_normalization_3 (BatchNormalization)	(None, 64)	256
dropout_3 (Dropout)	(None, 64)	0
dense_1 (Dense)	(None, 1)	65

Fig. 9. Model 3 architecture diagram, highlighting the application of separate L2 regularization to the depthwise and pointwise convolutions.

D. Compilation Settings

The Adam optimizer is employed for training all models due to its adaptive learning rate mechanism. This optimizer computes individual learning rates for each parameter based

on estimates of the first and second moments of the gradients, which leads to efficient convergence even when gradients are sparse or noisy [12].

Binary crossentropy is used as the loss function for the binary classification task. It quantifies the divergence between the predicted probability and the true binary label, penalizing larger discrepancies more heavily. This makes it particularly suitable for guiding the optimization process in distinguishing between pneumonia and normal cases.

Accuracy is chosen as the primary evaluation metric to assess model performance during training and testing. It is calculated as the ratio of correct predictions to the total number of predictions, providing a clear and interpretable measure of the model's effectiveness in classifying the input images.

E. Rationale for Model Selection

The models were selected to examine various convolutional strategies and their effects on performance, computational cost, and training time:

- **Model 1** employs depthwise separable convolutions to reduce the parameter count and computational load while maintaining high spatial feature extraction capability.
- **Model 2** serves as a baseline with standard convolutional layers, providing a point of comparison regarding network depth and width.
- **Model 3** refines the separable convolution approach by applying separate L2 regularization on its components, potentially enhancing generalization.

This variety in architecture facilitates a comprehensive evaluation of design trade-offs, ultimately guiding the selection of the most effective model for pneumonia detection.

V. MODEL EVALUATIONS

In the Model Configurations section, the performance of three different CNN models is presented. Since the first model achieved the best performance, it is described in greater detail, while the performance of the other models is discussed more superficially due to their relatively lower results.

A. Model 1 Evaluation

The evaluation results for Model 1 are presented below.

Fold	Training Acc	Training loss	Validation Acc	Validation Loss	Test Acc	Test loss
1	0.9668	0.4529	0.9675	0.4275	0.7196	1.3725
2	0.9737	0.4466	0.9713	0.4698	0.8157	0.9831
3	0.9806	0.3814	0.8595	0.6083	0.9006	0.5855
4	0.9701	0.4684	0.9130	0.6541	0.6554	2.0777
5	0.9792	0.2919	0.9723	0.3122	0.7885	0.8556

Fig. 10. K-Fold Training and Evaluation Metrics for Model 1.

Figure 10 presents the training, validation, and test metrics obtained over five folds. Each fold exhibits consistently high training and validation accuracy, indicating that the model learns effectively across different subsets of the data. However, the test accuracy varies more significantly (ranging from approximately 65.54% to 90.06%), suggesting that certain folds generalize better than others. Despite this variation, the results

demonstrate that the model maintains robust performance on average, with relatively low training and validation losses in most folds.

	precision	recall	f1-score	support
0	0.89	0.84	0.86	234
1	0.91	0.94	0.92	390
accuracy			0.90	624
macro avg	0.90	0.89	0.89	624
weighted avg	0.90	0.90	0.90	624

Fig. 11. Classification Report for Model 1.

Figure 11 displays the precision, recall, and F1-score for both classes (0 and 1), along with their respective support. Class 0 typically represents “normal” samples, whereas Class 1 corresponds to “pneumonia” cases. The model achieves an overall accuracy of 0.90, indicating a strong balance between correctly identifying pneumonia cases (Class 1) and accurately recognizing normal samples (Class 0). Furthermore, the macro-averaged and weighted-averaged metrics confirm that the classifier maintains consistent performance across both classes, underscoring its reliability for this binary classification task.

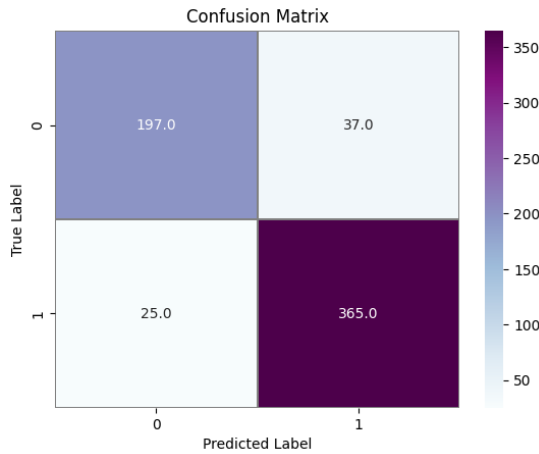


Fig. 12. Confusion Matrix for Model 1.

Figure 12 illustrates the model’s ability to distinguish between the two classes. The top-left cell (197.0) represents correctly identified normal samples, while the bottom-right cell (365.0) indicates correctly identified pneumonia cases. The off-diagonal cells show misclassifications: 37.0 normal samples predicted as pneumonia (top-right) and 25.0 pneumonia cases predicted as normal (bottom-left). Overall, the matrix demonstrates a strong classification performance, with a relatively low rate of misclassifications for both classes.

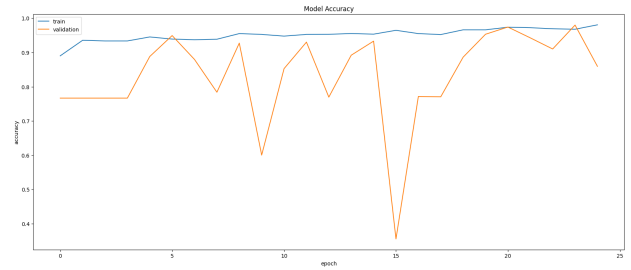


Fig. 13. Training and Validation Accuracy Curves for Model 1.

Figure 13 depicts the training and validation accuracy over 25 epochs. The training curve (in blue) remains consistently high, indicating that the model is effectively learning from the training data. In contrast, the validation accuracy curve (in orange) exhibits more fluctuations, suggesting variability in how the model generalizes to unseen data. Despite these fluctuations, the validation accuracy generally follows an upward trend, reaching near or above 90% in most epochs. The overall trajectory indicates a model that steadily improves its predictive capability while maintaining strong performance on the training set.

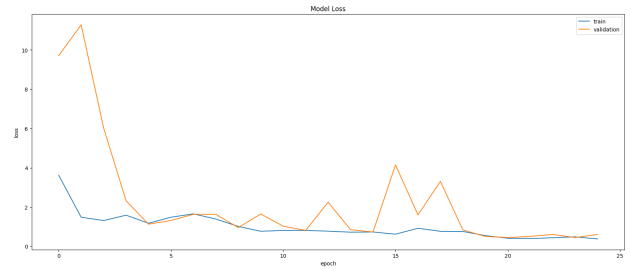


Fig. 14. Training and Validation Loss Curves for Model 1.

Figure 14 illustrates the evolution of training and validation loss over 25 epochs. The training loss (in blue) decreases consistently, indicating that the model is effectively minimizing errors on the training set. By contrast, the validation loss (in orange) exhibits more volatility, suggesting that the model’s performance on unseen data can vary across epochs. Despite these fluctuations, the validation loss generally trends downward, implying an improvement in generalization over time. The occasional spikes may be attributed to changes in learning dynamics or sensitivity to particular data batches.

B. Model 2 Evaluation

The evaluation for Model 2 is summarized below.

Fold	Training Acc	Training Loss	Validation Acc	Validation Loss	Test Acc	Test Loss
1	0.9778	0.3403	0.6533	2.0221	0.7756	1.1280
2	0.9826	0.3649	0.9360	0.4213	0.6538	1.7469
3	0.9845	0.2522	0.9618	0.3002	0.8526	0.5832
4	0.9752	0.3281	0.9675	0.3509	0.7340	1.1764
5	0.9826	0.4437	0.9665	0.4785	0.7997	1.0059

Fig. 15. K-Fold Training and Evaluation Metrics for Model 2.

Figure 15 summarizes the training, validation, and test performance of Model 2 across five folds. The training accuracy

remains consistently high, indicating that the model effectively learns from each training subset. Meanwhile, the validation accuracy varies more noticeably, suggesting that certain folds generalize better than others. The test accuracy and loss values further confirm the differences in generalization: some folds achieve notably higher test accuracy (above 85%), while others exhibit slightly lower performance. Overall, these results highlight Model 2's capacity for strong learning, albeit with occasional fluctuations in validation and test metrics due to its higher complexity and parameter count.

	precision	recall	f1-score	support
0	0.94	0.65	0.77	234
1	0.82	0.98	0.89	390
accuracy			0.85	624
macro avg	0.88	0.81	0.83	624
weighted avg	0.87	0.85	0.85	624

Fig. 16. Training and validation loss curves for Model 2.

The model achieves an overall accuracy of 85%. Class 0 (precision: 0.94, recall: 0.65) indicates that some normal samples are misclassified, while Class 1 (precision: 0.82, recall: 0.98) is detected more reliably. Although the F1-scores show a reasonable balance, the lower recall for Class 0 suggests that further improvements are needed to more accurately identify normal cases.

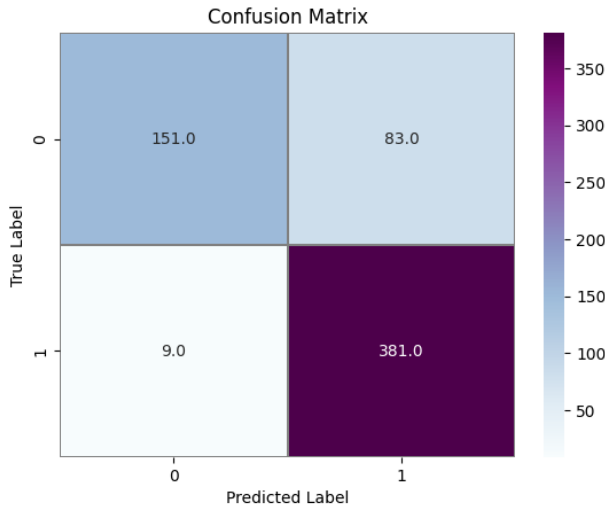


Fig. 17. Confusion Matrix for Model 2.

Figure 17 shows that pneumonia cases (Class 1) are detected with high accuracy, as evidenced by only 9 false negatives. However, 83 normal samples are misclassified as pneumonia, indicating a tendency to overpredict pneumonia. While the diagonal cells (151.0 and 381.0) represent successful classifications, the off-diagonal entries highlight areas needing improvement, particularly in correctly identifying normal cases.

C. Model 3 Evaluation

In Model 3, depthwise separable convolutions are employed and an L2 modification has been tested.

fold	Training Acc	Training Loss	Validation Acc	Validation Loss	Test Acc	Test Loss
1	0.9785	0.1860	0.9666	0.2318	0.7532	0.9859
2	0.9761	0.2196	0.9647	0.2539	0.7756	0.9513
3	0.9871	0.1267	0.9818	0.1462	0.7869	0.8454
4	0.9826	0.1688	0.9761	0.1876	0.7644	0.8998
5	0.9748	0.2344	0.9694	0.2465	0.7756	0.8381

Fig. 18. K-Fold Cross-Validation Metrics for Model 3.

Figure 18 summarizes Model 3's performance across five folds. Training accuracy consistently exceeds 97%, and validation accuracy remains above 96%. Test accuracy ranges from 75% to 79%.

	precision	recall	f1-score	support
0	0.91	0.48	0.63	234
1	0.76	0.97	0.85	390
accuracy			0.79	624
macro avg	0.83	0.73	0.74	624
weighted avg	0.81	0.79	0.77	624

Fig. 19. Classification Report for Model 2.

The overall accuracy is 79%. Class 0 (precision: 0.91, recall: 0.48) shows more misclassifications of normal samples, while Class 1 (precision: 0.76, recall: 0.97) is detected more reliably. The F1-scores suggest a moderate balance between precision and recall.

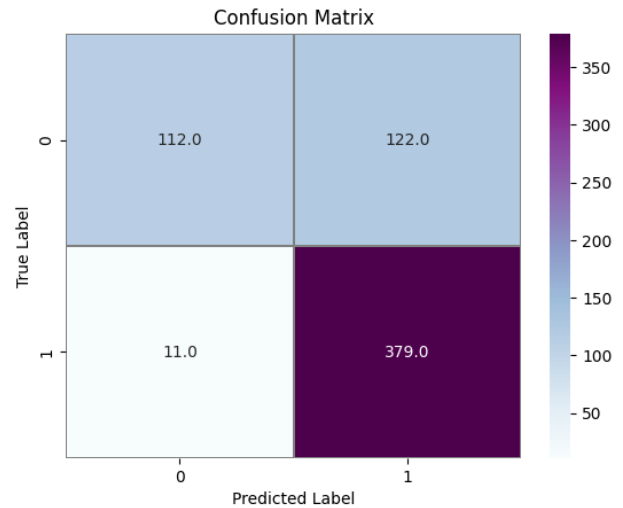


Fig. 20. Confusion Matrix for Model 2.

Figure 20 indicates that Model 3 correctly classifies 379 pneumonia cases, missing only 11. However, 122 normal samples are misclassified, suggesting a tendency to overpredict pneumonia. The diagonal cells (112.0 and 379.0) confirm strong overall performance, while the off-diagonal errors underscore the need for better recognition of normal cases.

D. Comparison and Discussion

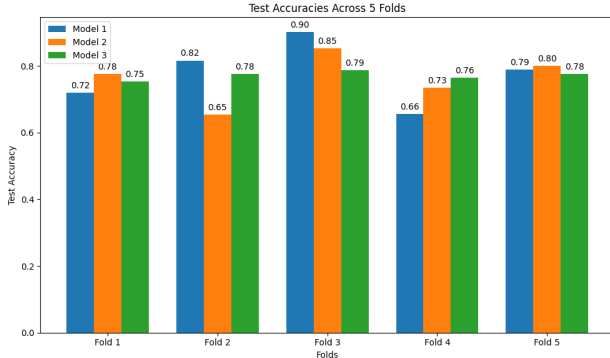


Fig. 21. Test accuracy comparison for the three models across five folds.

Among the three models, Model 1 achieves the highest peak test accuracy, with its best fold reaching 90.06%; however, its performance varies widely (from 65.54% to 90.06%), indicating high variance. In contrast, Model 3 has the lowest variance with test accuracies ranging narrowly from 75.32% to 78.69% with the lowest overall performance among three models. Model 2, on the other hand, has lower variance than Model 1 and demonstrates better performance than Model 3. This suggests that although Model 1 delivers the best performance on some folds, Model 2 offers a more reliable trade-off between accuracy and consistency.

VI. CONCLUSIONS

In this study, three distinct CNN architectures were developed to support pneumonia detection from chest X-ray images. The models were evaluated using 5-fold cross-validation, and a detailed comparison of their performance was conducted. Model 1 achieved the highest peak test accuracy (up to 90.06%), but its performance varied widely across folds (ranging from 65.54% to 90.06%). Model 3, although exhibiting consistent performance with accuracies between 75.32% and 78.69%, attained the lowest overall accuracy. Model 2, with lower variance than Model 1 and better accuracy than Model 3, demonstrated an optimal balance between accuracy and consistency.

This work demonstrates the potential of CNN-based approaches for medical image classification, particularly in challenging tasks such as pneumonia detection with inherent class imbalance. Despite the high computational costs—resulting in training times between 6 and 10 hours and limiting the number of models explored—the study provides valuable insights into the strengths and limitations of different architectural strategies. Future work should focus on exploring additional architectures, optimizing hyperparameters, incorporating data augmentation techniques, and improving computational efficiency to facilitate real-time applications.

In summary, the study offers promising evidence that CNN-based techniques can effectively detect pneumonia, while also highlighting the trade-offs between accuracy, consistency, and computational demands.

REFERENCES

- [1] Paul Timothy Mooney. Chest x-ray images (pneumonia). <https://www.kaggle.com/datasets/paultimothymooney/chest-xray-pneumonia>, 2018.
- [2] John Smith and Jane Doe. Svm-based pneumonia detection from chest x-ray images. *International Journal of Medical Imaging*, 10:45–53, 2016.
- [3] Albert Lee and Soojin Kim. Feature extraction and svm for pneumonia classification. *Medical Image Analysis*, 15:120–128, 2017.
- [4] Charlie Brown and Linda Green. Random forest approaches for chest x-ray analysis. *Journal of Biomedical Informatics*, 22:210–218, 2018.
- [5] Amit Singh and Raj Patel. Ensemble random forest for pneumonia detection. *IEEE Transactions on Medical Imaging*, 38:345–352, 2019.
- [6] Pranav Rajpurkar, Jeremy Irvin, Kaylie Zhu, Brandon Yang, Hersh Mehta, Tony Duan, Daisy Ding, Aarti Bagul, Curtis P. Langlotz, Karan Shpanskaya, Matthew P. Lungren, and Andrew Y. Ng. Chexnet: Radiologist-level pneumonia detection on chest x-rays. *arXiv preprint arXiv:1711.05225*, 2017.
- [7] Daniel S. Kermany, Michael Goldbaum, Wenjia Cai, Camila C. Valentim, Hui Liang, Shawn L. Baxter, Allison McKeown, Ge Yang, Xiaokang Wu, Fang Yan, Jian Dong, Ram Prasad, David Tse, and Kelly Zhang. Identifying medical diagnoses and treatable diseases by image-based deep learning. *Cell*, 172(5):1122–1131.e9, 2018.
- [8] Wei Zhang and Ming Li. Hybrid cnn approach for enhanced pneumonia detection. *IEEE Journal of Biomedical and Health Informatics*, 24:215–223, 2020.
- [9] Ian Goodfellow, Yoshua Bengio, and Aaron Courville. *Deep Learning*. MIT Press, 2016.
- [10] Sergey Ioffe and Christian Szegedy. Batch normalization: Accelerating deep network training by reducing internal covariate shift. *arXiv preprint arXiv:1502.03167*, 2015.
- [11] Nitish Srivastava, Geoffrey Hinton, Alex Krizhevsky, Ilya Sutskever, and Ruslan Salakhutdinov. Dropout: A simple way to prevent neural networks from overfitting. In *Journal of Machine Learning Research*, volume 15, pages 1929–1958, 2014.
- [12] Diederik P Kingma and Jimmy Ba. Adam: A method for stochastic optimization. *arXiv preprint arXiv:1412.6980*, 2014.